vRecord: A User Study Dataset on Video Watching Behavior

Jiayu Zhu
jiayuz6@illinois.edu
University of Illinois Urbana-Champaign
Urbana, Illinois, USA

Yih-Chun Hu
yihchun@illinois.edu
University of Illinois Urbana-Champaign
Urbana, Illinois, USA

Abstract

Today's Internet traffic is dominated by video content, which uses adaptive bitrate (ABR) algorithms and buffers to improve streaming quality. However, many traditional ABR designs mainly consider and evaluate linear viewing, overlooking common interactions such as seeking, replaying, and short skips. In traditional ABRs, these nonlinear interactions lead to buffer waste and increased stalls. To study these effects, we built an interactive streaming platform that supports progress-bar seeking, fast-forward/replay, and keyboard shortcuts for fixed-distance skipping. We further implement a stall-minimizing seek mechanism that trades seek accuracy for reduced post-seek stalls, allowing us to quantify how stall reduction changes user behavior. Using 5448 viewing traces, we observe strongly heavy-tailed viewing dynamics: while most sessions exhibit limited interaction, a small fraction accounts for a disproportionate number of seeks. On average, users seek 5.91 times forward and 0.76 times backward per session, and the stall-minimizing mechanism experiences 10.9% more jumps. We also observe early-abandonment patterns in a subset of traces. Overall, our results show that user interaction is a first-order factor that is often missing from modern streaming pipeline design and evaluation.

CCS Concepts

• **Information systems** → **Multimedia streaming**; • **Human-centered computing** → **Empirical studies in HCI**; • **Networks** → *Network measurement*; Application layer protocols.

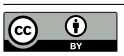
Keywords

Video Streaming, Quality of Experience, User Behavior

ACM Reference Format:
Jiayu Zhu and Yih-Chun Hu. 2026. vRecord: A User Study Dataset on Video Watching Behavior. In *Proceedings of the 34th ACM International Conference on Multimedia (MM '26), November 10–14, 2026, Rio de Janeiro, Brazil*. ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3767308.3838679

Ethical Considerations

This study is IRB-approved. We use informed consent, do not personally identify users and do not collect more than industry-standard telemetry about video playback. The only data we log that is likely to be privacy-sensitive are IP addresses, which we purge within 3 months of collection. We allow users to opt-out of the study and view the video of their choice on a platform without any telemetry. This study is restricted to users in United States. We expect most of the users to be students of our introduction to computing course.



This work is licensed under a Creative Commons Attribution 4.0 International License.
MM '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2213-4/2026/11
https://doi.org/10.1145/3767308.3838679

1 Introduction

Video streaming accounts for roughly 65% of global Internet traffic [17]. Consequently, significant research effort has been devoted to the improvement of video streaming, including work in quality-of-experience (QoE) metrics [12] and adaptive bitrate (ABR) streaming techniques [20, 5, 24, 19]. Yet most existing QoE definitions and ABR evaluation methodologies implicitly assume *full, linear viewing*, where users watch a video end-to-end without seeking, replaying, or skipping [14, 15]. In practice, users frequently interact with the player, and these interactions can substantially change both QoE metrics and system behavior. Despite this, ABR algorithms are still commonly evaluated in a linear play-through environment, and today's QoE metrics often fail to capture interaction-driven behaviors during playback, and thus cannot explain the impact of user interaction on QoE. This gap persists largely because actual user playback traces are difficult to obtain at scale due to deployment cost, privacy and consent constraints, and the need for instrumented players in realistic viewing settings [25].

To address this limitation, our goal is to build the largest open database of video playback traces from real-world educational streaming. We collect 5448 traces from our interactive video streaming platform from January 18, 2025 through July 19, 2026 and they are released through Github [29]. Each trace includes client-side measurements of browser and operating system information, time since page load, frames played, viewing intervals, stall time, and the video identifier. By enabling large-scale analysis and reproducible evaluation under realistic interaction patterns, our dataset supports the development and evaluation [27] of QoE metrics and ABR methods that reflect how users actually watch videos.

Contributions. Our primary contributions are:

- A database, consisting of 5448 playback traces from real users, released through Github: https://github.com/jimmyzhu12/vRecord
- We host a video streaming platform that records actual user behavior and actively updates the database.
- We studied a potential strategy for reducing stall-time on user-seek, and analyzed its impact on user behavior.

2 Implementation and data collection

We built a custom video-streaming website that we host on our own server. On this website, we hosted course videos for adaptive streaming. Each course video was prepared using a standardized encoding pipeline, then packaging it in MPEG-DASH format [19]. We implemented a streaming UI that mirrors common commercial player functionality to encourage natural viewing behavior. We also modified the Dash.js [7] client to implement our stall reduction

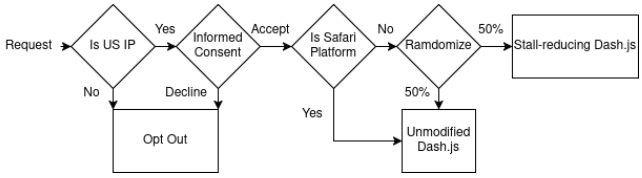


Figure 1: The participant intake procedures per IRB approval.

strategy, which trades seek accuracy for reduced seek-induced stalling. Finally, we instrumented the client to collect per-playback telemetry, randomize between the unmodified Dash.js pipeline and our stall-reduction pipeline, and transmit metrics for logging to a backend database.

Our user intake process, as illustrated in Figure 1, was designed to meet IRB requirements. First, to comply with data protection laws (e.g. GDPR), we use an IP geolocation database to exclude non-US IP addresses; these users are sent to a separate section of the site that does not collect metrics; we label this section "Opt Out" as it is also used to serve content to users who opt out of the experiment. Next, we show a banner on the bottom of the video selection screen, which describes the experiment, including randomization, data collection, and risks; at this point, a user may opt out of the experiment, and be sent to the "Opt Out" flow. If the user accepts the Informed Consent, we then evaluate whether the user is rendering the video on Safari. We check for Safari usage because the stall-reduction pipeline introduces non-contiguous segments to the Media Source Buffer, and Safari tends to crash when too many non-contiguous segments are added to its Media Source Buffer; consequently, we assign Safari users to the unmodified Dash.js, bypassing the randomization step. (Even third-party browsers on iOS often use Safari to comply with Section 2.5.6 of the App Review Guidelines [1].) Finally, we randomize users between our unmodified Dash.js and our stall-reducing version.

2.1 Video Preparation

We offered a total of 61 pre-lecture videos for ECE 220 *Computer Systems & Programming* offered at the University of Illinois at Urbana-Champaign. The videos range in duration from 1:29 (89 seconds) to 26:38. To ensure a fair comparison and standardized processing, we uploaded all videos to YouTube and then downloaded the transcoded versions in 1080p using `yt-dlp` [8], so that every video underwent the same YouTube encoding pipeline. We subsequently re-encoded the content using FFmpeg [10] at the bitrates recommended for YouTube ingest [26], enforcing a keyframe interval of 2 s, and generated three output resolutions: 480p, 720p, and 1080p. Finally, we packaged the videos for MPEG-DASH and delivered them using Dash.js [7], a widely used open-source media player that provides reliable DASH playback in modern browsers. This setup allows participants from virtually all platforms and browser-based devices to take part in the study. Since the videos are instructional videos made by the research group, we can confirm that we hold full copyright for all videos used in the study.

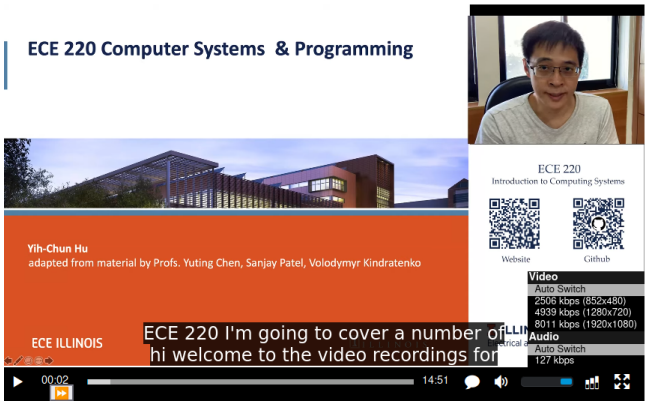


Figure 2: An example of the interactive video player.

2.2 UI Implementation

To capture natural viewing behavior, we implemented several key features so that our website closely resembles modern streaming platforms (e.g., YouTube). We provided full video transcripts, allowing users to follow the content even when audio is muted. We also added a one-click 8-second skip-forward control and mapped it to the keyboard arrow keys for convenient navigation. For video-quality selection, users can either rely on adaptive bitrate streaming (Dash.js's default ABR algorithm) [20] or manually select a fixed resolution (and thus bitrate). Finally, a full-screen mode was available to improve the overall viewing experience.

2.3 Stall-Reduction Strategy

In previous work, we designed a strategy that subdivides standard MPEG-DASH segments into smaller sub-segments (similar to low-latency DASH) and prebuffers the beginnings of each segment to reduce seek-induced stalls down to a few ms (time required to push the prefetched chunks into Media Source Buffer). To ensure that seeks land exactly on prefetched content, we modified the seek bar to only allow seeking to segment boundaries. To avoid unnecessary buffer waste from prefetching, we also reduce the maximum buffer length. This strategy primarily benefits users with sufficient network capacity, and may negatively affect users whose networks cannot prefetch in time. Therefore, by default we use the unmodified Dash.js ABR and implemented a conservative policy for switching to the stall-reduction strategy: the strategy is enabled only for users with strong network performance, while users with weaker networks continue to stream regular MPEG-DASH segments, ensuring no change to their QoE. We further describe this strategy and the implementation challenges and details in our previous paper StallFreeSeek [28].

2.4 Data Collection

Our pipeline for participant intake is shown in Figure 1. After consent and assignment, each client records the following data: browser and operating system, time since page load, frames rendered, a per-session UUID, viewing intervals, stall duration, and the video identifier. We use `platform.js` [3] to detect the browser and

OS. We compute time since page load from a global timestamp captured via `Date.now()`. We track frames rendered using the HTML video element's `requestVideoFrameCallback` [11]. We generate a session UUID with `uuidjs` [23]. We derive viewing intervals and stall duration from HTML media state-change events. The client transmits all metrics to the server once per second, where they are stored in a database.

Besides these records that the server receives from the client, the server also keeps standard server logs; we remove the IP addresses from these logs in accordance with our IRB protocol. Each video session is uniquely identified using the generated UUID, allowing us to retrieve statistics about buffer health and the network from the server logs. Detailed description of database attributes is in Table 1.

There are a total of 2963 traces on unmodified Dash.js, and a total of 2485 traces on Dash.js with the stall-reduction strategy. For a fair comparison, when plotting graphs, we removed the traces from Apple platform that does not support stall-reduction strategy, resulting in 2509 unmodified Dash.js traces. Also, only traces that render at least one video frame are recorded into the database.

Parameter	Description
browserInfo	string. Browser identity from User-Agent.
osInfo	string. Operating system identity from User-Agent.
timeSincePageLoad	long. Seconds since webpage loaded.
modified	integer. Regular vs stall-reducing.
uuid	string. Unique ID of the playback session.
stallDiffsString	string. Array of recorded stalls.
videoId	string. The ID of the video.
timestamp	datetime. Playback session start.
totalWatchTime	long. The total video watch time.
watchRangeFrames	string. Array of the range of frames watched.
watchRangeSeconds	string. Array of the range of seconds in video watched.
apiCatchWatchSegments	string. Array of the exact API-given playback frames.

Table 1: Detailed breakdown of attributes in the dataset. The dataset can be reviewed through Github [29].

2.5 Browser and Operating Systems

We tabulate the number of users using each browser and operating system information in Table 2. These users are subdivided by the version of Dash.js used: the unmodified version or the stall-reduction version. Most users access our site on desktops, and among desktop users, most use Windows. Among our traces, 3606 views are from Google Chrome, whereas Microsoft Edge, Firefox and Safari together only account for 1436 traces. These browser shares are consistent with the market share rate of all browsers as shown in GlobalStats [21].

	Win	MacOS	Ubuntu	Android	Linux	iOS	Chrome OS
Chrome	1225;1200	535;524	0;0	0;0	58;55	0;0	2;7
Microsoft Edge	279;256	12;19	0;0	0;0	0;0	0;0	0;0
Firefox	168;137	16;22	48;61	0;0	8;6	0;0	0;0
Safari	0;0	323;0	0;0	0;0	0;0	81;0	0;0
Chrome Mobile	0;0	0;0	0;0	18;44	0;0	49;0	0;0
Opera	124;132	0;0	0;0	0;0	0;0	0;0	0;0
Samsung Internet	0;0	0;0	0;0	15;20	0;0	0;0	0;0
Android Browser	0;0	0;0	0;0	1;1	0;0	0;0	0;0
Electron	0;1	0;0	0;0	0;0	0;0	0;0	0;0
Firefox for iOS	0;0	0;0	0;0	0;0	0;0	1;0	0;0

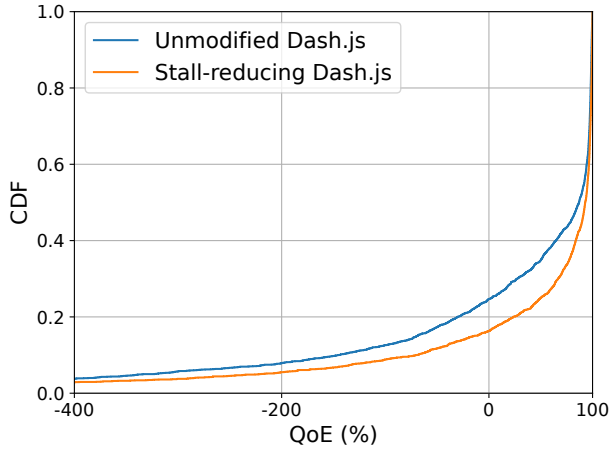
Table 2: Trace counts by browser and OS. The first number in each pair reflects the number of unmodified Dash.js users with this configuration; the second number, the number of stall-reduction users.

2.6 QoE

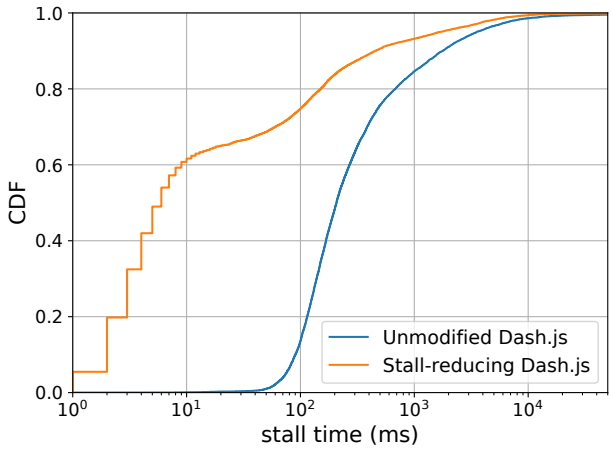
We present the QoE score using a definition similar to the one used in SODA [5]. Our score is a linear combination of three components: mean utility, rebuffering ratio, and switch rate. Mean utility reflects video quality as the logarithm of the bit-rate. Rebuffering ratio is the ratio of stall time to view duration, i.e., $\rho_{\text{rebuf}} = \frac{T_{\text{rebuf}}}{T}$. Switch rate is the number of bitrate switches divided by view duration in seconds, i.e., $p_{\text{switch}} = \frac{N_{\text{switch}}}{T}$. The QoE score is simply a linear combination of the three QoE components: $\text{QoE} = \bar{v} - \beta \cdot \rho_{\text{rebuf}} - \gamma \cdot p_{\text{switch}}$. We choose $\beta = 10$ and $\gamma = 1$ just like in SODA [5]. We set the weights to these values as stall is undesired and reduces engagement [9, 12]. We calculated a QoE score for each video session, and plot the CDF for both versions of Dash.js in Figure 3a. In our dataset, nearly half of the video sessions have a high QoE, likely due to good network conditions and limited seeking. Furthermore, QoE drops off more rapidly for the unmodified Dash.js, showing that the stall-reducing Dash.js improves QoE; other graphs will show that this QoE improvement arises from reduced stall time. While most of the sessions have decent QoE scores, some of the sessions have negative QoE scores. This is because of low user retention caused by early abandonment and high seek numbers. For example, some sessions contain more than 20 seeks, resulting in excessive seek-related stall time.

2.7 Stall Time

We define *stall time* to be the time elapsed between when the playback buffer is depleted, thus causing the video to pause, and when the buffer fills enough for the ABR to allow resumption of playback. We consider all stalls, whether induced by network conditions or user actions. We extract every stall event across all video sessions and plot the CDF of these stall times in Figure 3b. This figure shows that 60% of stalls in the stall-reducing Dash.js are less than 10 ms, whereas the 60th percentile stall time with unmodified Dash.js is 270 ms. We highlight that prebuffered segments that the stall-reducing Dash.js plays back in response to a seek often occurs faster than 1 RTT because the stall-reducing Dash.js retrieves that segment from a javascript variable (rather than over the network) and inserts it into the Media Source Buffer for playback; further details about this approach can be found in our paper about our stall-reduction strategy [28]. These results show that the stall-reduction strategy substantially reduces stall time compared to unmodified

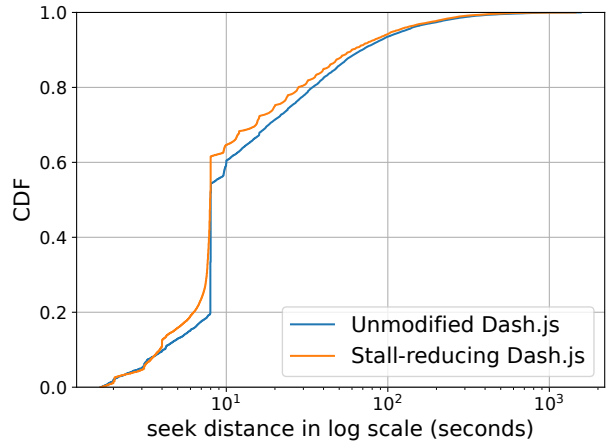


(a) CDF of QoE for unmodified Dash.js and Stall-Reduction Dash.js.

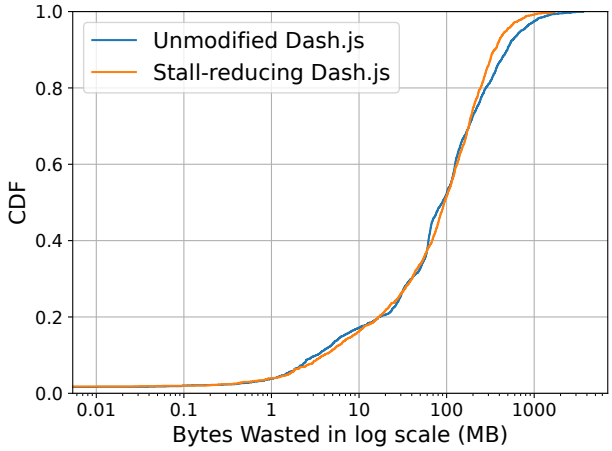


(b) CDF of single stall time for unmodified Dash.js and Stall-Reduction Dash.js.

Figure 3: QoE and seek performance comparison for unmodified Dash.js and Stall-Reduction Dash.js.



(a) CDF of seek distance for unmodified Dash.js and Stall-Reduction Dash.js.



(b) CDF of bytes wasted for unmodified Dash.js and Stall-Reduction Dash.js.

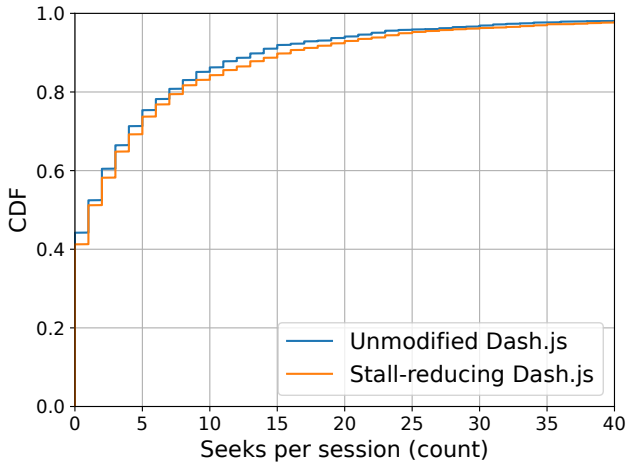
Figure 4: Seek distance and buffer waste comparison for unmodified Dash.js and Stall-Reduction Dash.js.

Dash.js, confirming that the strategy achieves its intended effect and enabling us to study how reduced stalling influences user behavior.

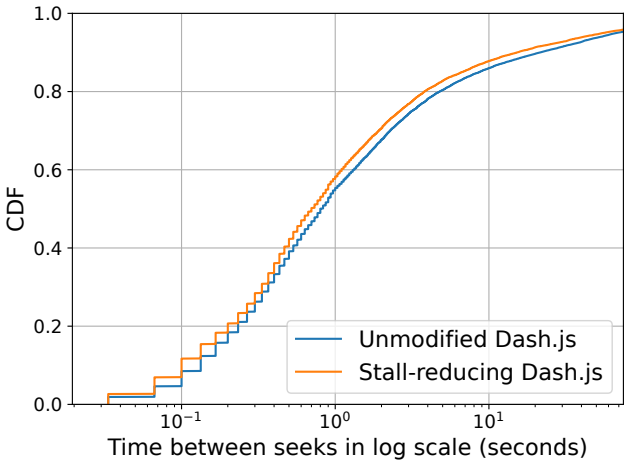
2.8 Seek Behavior

Filtering. Our client-side code uses the HTMLVideoElement API [11] to determine the current video frame being played back; however, this API may not capture every frame that is being played due to performance differences across devices and limitations of the API. Furthermore, our design uses a 436 pixel wide seek bar, limiting the precision of a user-initiated seek. As a result, we do not count skips of less than 50 frames to be an actual seek.

Seek Distance. Seek distance is defined as the playback time offset of a seek operation (i.e., the number of seconds the user jumps forward or backward). We extract every seek event across all video sessions and plot the empirical CDF of seek distance in Figure 4a. This figure shows that most seeks are local. For long-distance seeks, the seek-length distributions of unmodified Dash.js and the stall reduction configuration are similar, suggesting that these jumps are primarily content-driven. For shorter jumps, unmodified Dash.js shows a significant number of jumps of 8 seconds due to the keyboard shortcuts, whereas in the stall-reducing Dash.js, the seek-time destination is rewritten to the beginning of the nearest chunk, smoothing out the seek distance. The spike at 8 seconds is still present in stall-reducing Dash.js because some users will perform multiple keyboard-driven seeks.

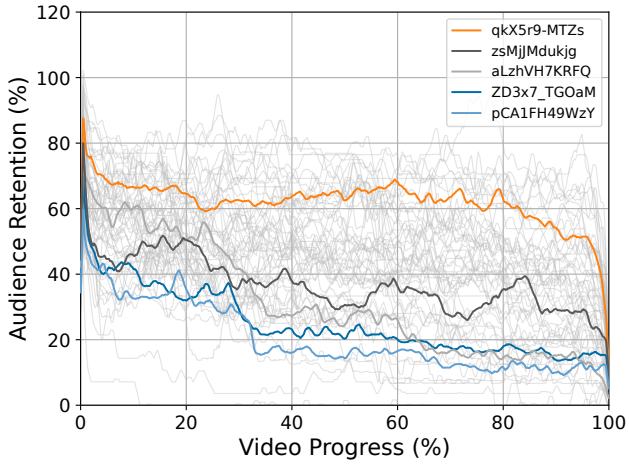


(a) CDF of seeks per session.

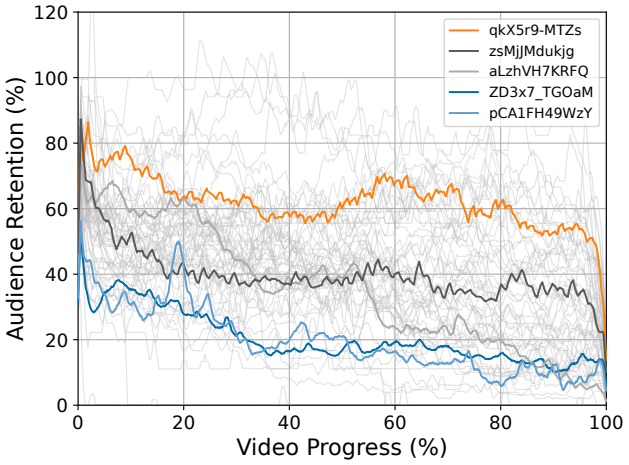


(b) CDF of time between seeks.

Figure 5: Seek behavior comparison for unmodified Dash.js and Stall-Reduction Dash.js.



(a) unmodified Dash.js users.



(b) Stall-Reduction Dash.js users.

Figure 6: User retention curves on all videos with top 5 watched videos highlighted.

Number of seeks. We aggregate the number of seeks for each video session and plot the CDF in Figure 5a. Most sessions contain at least one seek, indicating that seeking is common in real-world viewing. Compared to unmodified Dash.js, stall-reduction Dash.js exhibits a slightly higher number of seeks per session overall. This association is consistent with reduced post-seek stall time lowering the cost of seeking, though our study design does not allow us to establish a causal relationship between the stall-reduction strategy and increased seeking. However, the fraction of sessions with zero seeks is the same, suggesting that among users who seek at all, higher stall time is correlated with fewer additional seeks.

Time between seeks. Time between seeks is calculated based on the number of frames played between consecutive seek operations. We aggregate these measurements and plot their empirical CDF in Figure 5b. Our results suggest that stall-reduction Dash.js users select shorter inter-seek times for intervals below 75 seconds, except for seek intervals of 8 seconds, which is the skip time of the provided keyboard shortcut. We hypothesize that by reducing stall duration, the stall-reduction strategy lowers the perceived cost of seeking and thus encourages more frequent user navigation. For inter-seek intervals above 75 seconds, seeking behavior is more likely driven by the video content itself rather than the buffering strategy.

2.9 Buffer Waste

Buffer waste is calculated by combining server logs with frame-level playback information. The server logs provide the set of frames (and corresponding bytes) fetched for each session; we compare the

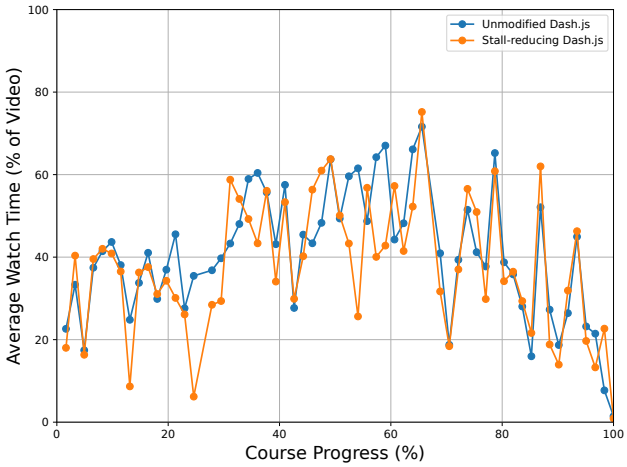


Figure 7: Average watch time across the lecture series.

downloaded frames against the frames actually played, and compute wasted data as the frames (bytes) that were downloaded but never rendered. Figure 4b plots the empirical CDF of buffer waste for unmodified Dash.js and the stall-reduction Dash.js. Overall, the stall-reduction configuration exhibits lower buffer waste, likely due to the modified seek-bar behavior and a reduced linear-buffer size, which limit overfetching. (Unmodified Dash.js will fetch up to 60 seconds of buffer, whereas the stall-reduction Dash.js limits itself to 12 seconds.) However, around the 60th percentile of buffer waste, unmodified Dash.js shows slightly lower waste. We hypothesize that this crossover reflects a trade-off: unmodified Dash.js has longer linear buffers, whereas stall-reduction Dash.js performs prefetching; when there are fewer skips, both systems experience less waste, but this tradeoff tilts in favor of unmodified Dash.js.

2.10 User Retention

Retention per Video. To reconstruct audience retention curves, we first partition each video timeline into 10,000 equally-sized time bins. For every session, we obtain the list of watched frame segments and compute, for each bin, the amount of watched time that overlaps the bin. The retention curve is then formed by summing these per-bin overlaps across all sessions and normalizing by the bin duration. At each point, we plot the average of that bin with its neighboring 100 bins to make local spikes and drop-offs easier to identify. Retention can exceed 100% because users may replay portions of the video, causing the total watched time for a bin to exceed its duration. We plot retention curves for all videos and highlight the top five most-played titles. Figure 6a shows the curves under unmodified Dash.js: most sessions exhibit early abandonment, with occasional spikes later in the video, likely corresponding to salient moments. Figure 6b shows the curves under the stall reduction configuration. Because stall-reduction Dash.js rewrites seek times, retention patterns of stall-reduction users are less smooth.

Retention per Course Progress. To evaluate user retention across the lecture series, we compute the average watch time per video from frames played per session and normalize it by video length to obtain a retention ratio. Each video is mapped to its position in the course progression. Figure 7 shows several spikes, suggesting that differences in video content influence retention and that users do not necessarily watch videos from beginning to end.

3 Related Work

Extensive prior work has examined user behavior in video streaming. Using user retention curves from video streaming platforms, a recent paper [4] shows that both seeking intensity and retention vary substantially across video content and across countries. Furthermore, rebuffering reduces engagement [9, 12]. Seeking behavior has also been shown to influence view counts and retention [15].

Video quality of experience (QoE) has been studied extensively, and many algorithms have been proposed to improve QoE. Studies on the relative impact of QoE metrics on subjective Quality of Experience [2] show that rebuffering has a very negative impact, while bitrate switches are less important. A recent adaptive bitrate algorithm, SODA [5], reflects this priority by placing strong emphasis on rebuffering ratio in their QoE metrics.

Previous work has also considered video consumption of actual Internet users. Puffer [24] builds a server that streams live TV to users. It records essential metrics such as rtt and video chunk delivery rate. We go one step further by streaming VoD content that allows users to watch nonlinearly, unlike live TV. Some existing work [16] analyzes commercial video behavior but does not provide access to the database, and thus lacks detailed information on video sessions. Without detailed video session information, researchers cannot evaluate their algorithms on real-world traces.

Prior work has explored reducing seek-related stall time. Dashlet [13] focuses on prefetching across videos, while HotDash [18] uses learning-based logic to prefetch likely hot regions within a video. In contrast, our stall-reduction Dash.js, SFS [28] aims to provide seamless seeking across the entire video rather than being limited to a small set of hotspots, which exploits faster network connections and robust infrastructure [6], supported by ongoing efforts to systematically test and harden deployments [22]. Our dataset provides an additional resource to support, evaluate, and validate these approaches, and new methods for reducing seek-related stall time.

4 Conclusion

In this paper, we collect 5448 user-behavior traces of educational videos through a user study on a server that mimics commercial Video-on-Demand platforms. By analyzing these traces, we uncover a wide range of viewing behaviors that are relevant to ABR evaluation. Our results show that users frequently abandon videos early, and seek both forward and backward while watching videos. User behavior also differs slightly under a stall-reduction strategy. Beyond enabling detailed studies of user behavior, the resulting dataset also provides a practical benchmark for testing and validating video streaming algorithms, such as ABR algorithms, and evaluation metrics. This dataset can also be used as a validation set for machine learning approaches.

References

[1] Apple Inc. 2025. App review guidelines. Apple Developer. (Nov. 13, 2025). Retrieved Jan. 15, 2026 from https://developer.apple.com/app-store/review/guidelines/.

[2] Christos George Bampis, Zhi Li, Anush Krishna Moorthy, Ioannis Katsavounidis, Anne Aaron, and Alan Conrad Bovik. 2017. Study of temporal effects on subjective video quality of experience. *IEEE Transactions on Image Processing*, 26, 11, 5217–5231. doi:10.1109/TIP.2017.2729891.

[3] BestieJS. 2024. Platform.js: a platform detection library. Version 1.3.6. GitHub repository (archived). (Apr. 11, 2024). Retrieved Jan. 13, 2026 from https://github.com/bestiejs/platform.js.

[4] Bo-Rong Chen, Jiayu Zhu, Yanxin Jiang, and Yih-Chun Hu. 2024. Vretention: a user viewing dataset for popular video streaming services. In *Proceedings of the 15th ACM Multimedia Systems Conference* (MMSys '24). Association for Computing Machinery, Bari, Italy, 313–318. ISBN: 9798400704123. doi:10.1145/3625468.3652175.

[5] Tianyu Chen, Yiheng Lin, Nicolas Christianson, Zahaib Akhtar, Sharath Dharmaji, Mohammad Hajiesmaili, Adam Wierman, and Ramesh K. Sitaraman. 2024. Soda: an adaptive bitrate controller for consistent high-quality video streaming. In *Proceedings of the ACM SIGCOMM 2024 Conference* (ACM SIGCOMM '24). Association for Computing Machinery, Sydney, NSW, Australia, 613–644. ISBN: 9798400706141. doi:10.1145/3651890.3672260.

[6] CTIA. 2025. 5G in America. Online; accessed Jan 31, 2025. CTIA - The Wireless Industry. (2025). https://www.ctia.org/the-wireless-industry/5g-in-america.

[7] DASH Industry Forum. 2024. dash.js: A Reference Client Implementation for the Playback of MPEG DASH via JavaScript and Compliant Browsers. https://github.com/Dash-Industry-Forum/dash.js. Accessed Jan 31, 2025. (2024).

[8] yt-dlp contributors. 2023. Yt-dlp: a command-line video downloader. GitHub repository. Accessed Jan 22, 2025. (2023). https://github.com/yt-dlp/yt-dlp.

[9] Florin Dobrian, Vyas Sekar, Asad Awan, Ion Stoica, Dilip Joseph, Aditya Ganjam, Jibin Zhan, and Hui Zhang. 2011. Understanding the impact of video quality on user engagement. In *Proceedings of the ACM SIGCOMM 2011 Conference* (SIGCOMM '11). Association for Computing Machinery, Toronto, Ontario, Canada, 362–373. ISBN: 9781450307970. doi:10.1145/2018436.2018478.

[10] FFmpeg Development Team. 2023. FFmpeg. Open source multimedia framework for decoding, encoding, transcoding, muxing, demuxing, streaming, filtering, and playing. Accessed Jan 21, 2025. (2023). https://ffmpeg.org.

[11] Thomas Guilbert. 2024. HTMLVideoElement.requestVideoFrameCallback(). Draft Community Group Report. Accessed Jan 22, 2025. Web Platform Incubator Community Group (WICG). https://wicg.github.io/video-rvfc/.

[12] S. Shunmuga Krishnan and Ramesh K. Sitaraman. 2012. Video stream quality impacts viewer behavior: inferring causality using quasi-experimental designs. In *Proceedings of the 2012 Internet Measurement Conference* (IMC '12). Association for Computing Machinery, Boston, Massachusetts, USA, 211–224. ISBN: 9781450317054. doi:10.1145/2398776.2398799.

[13] Zhuqi Li, Yaxiong Xie, Ravi Netravali, and Kyle Jamieson. 2023. Dashlet: taming swipe uncertainty for robust short video streaming. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, (Apr. 2023), 1583–1599. ISBN: 978-1-939133-33-5. https://www.usenix.org/conference/nsdi23/presentation/li-zhuqi.

[14] Hyunwoo Nam, Kyung-Hwa Kim, and Henning Schulzrinne. 2016. Qoe matters more than qos: why people stop watching cat videos. In *IEEE INFOCOM 2016 - The 35th Annual IEEE International Conference on Computer Communications*. IEEE, San Francisco, CA, 1–9. doi:10.1109/INFOCOM.2016.7524426.

[15] Minsu Park, Mor Naaman, and Jonah Berger. 2021. A data-driven study of view duration on youtube. *Proceedings of the International AAAI Conference on Web and Social Media*, 10, 1, (Aug. 2021), 651–654. doi:10.1609/icwsm.v10i1.14781.

[16] Fan Qiu and Yi Cui. 2010. An analysis of user behavior in online video streaming. In *Proceedings of the International Workshop on Very-Large-Scale Multimedia Corpus, Mining and Retrieval* (VLS-MCMR '10). Association for Computing Machinery, Firenze, Italy, 49–54. ISBN: 9781450301664. doi:10.1145/1878137.1878149.

[17] Sandvine. 2023. Global Internet Phenomena Report. https://www.prnewswire.com/news-releases/sandvines-2023-global-internet-phenomena-report-shows-24-jump-in-video-traffic-with-netflix-volume-overtaking-youtube-301723445.html. Accessed April 23, 2023. (2023).

[18] Satadal Sengupta, Niloy Ganguly, Sandip Chakraborty, and Pradipta De. 2018. Hotdash: hotspot aware adaptive video streaming using deep reinforcement learning. In *2018 IEEE 26th International Conference on Network Protocols (ICNP)*. IEEE, Cambridge, UK, 165–175. doi:10.1109/ICNP.2018.00026.

[19] Iraj Sodagar. 2011. The mpeg-dash standard for multimedia streaming over the internet. *IEEE MultiMedia*, 18, 4, 62–67.

[20] Kevin Spiteri, Rahul Urgaonkar, and Ramesh K. Sitaraman. 2020. Bola: near-optimal bitrate adaptation for online videos. *IEEE/ACM Transactions on Networking*, 28, 4, 1698–1711. doi:10.1109/TNET.2020.2996964.

[21] StatCounter Global Stats. 2026. Browser market share worldwide. Retrieved Jan. 14, 2026 from https://gs.statcounter.com/browser-market-share.

[22] Sixu Tan, Zeyu Li, Zhutian Liu, Harsh Patel, and Zhaowei Tan. 2025. Automated model-based fuzzing for 5g o-ran. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking* (ACM MOBICOM '25). Association for Computing Machinery, Kerry Hotel, Hong Kong, Hong Kong, China, 201–215. ISBN: 9798400711299. doi:10.1145/3680207.3723469.

[23] uuidjs. 2025. Uuid: generate rfc-compliant uuids in javascript. Version 13.0.0. GitHub repository. (Sept. 8, 2025). Retrieved Jan. 13, 2026 from https://github.com/uuidjs/uuid.

[24] F. Y. Yan, H. Ayers, C. Zhu, S. Fouladi, J. Hong, K. Zhang, P. Levis, and K. Winstein. 2020. Learning in situ: a randomized experiment in video streaming. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*. USENIX. Santa Clara, CA, 495–511. ISBN: 9781939133137. https://www.usenix.org/conference/nsdi20/presentation/yan.

[25] YouTube. [n. d.] Youtube privacy settings to protect your data. Retrieved Jan. 13, 2026 from https://www.youtube.com/howyoutubeworks/privacy/.

[26] YouTube Support. 2025. Youtube recommended upload encoding settings. Online; accessed Jan 31, 2025. YouTube support page on video encoding recommendations. (2025). https://support.google.com/youtube/answer/1722171.

[27] Bei Zhao, Jinhui Song, Jiayu Zhu, and Yih-Chun Hu. 2026. Streamlens: an open-source abr simulation environment for multi-region buffering, seeking, and prefetching. In *Proceedings of the 34th ACM International Conference on Multimedia* (MM '26). Association for Computing Machinery, Rio de Janeiro, Brazil. doi:10.1145/3767308.3834749.

[28] Jiayu Zhu, Bo-Rong Chen, Jinhui Song, Yifang Zhang, Yih-Chun Hu, Tianrui Wang, and Sibo Zhang. 2026. Stallfreeseek: exploiting better network conditions to improve video seek experience. *Proc. ACM Netw.*, 4, CoNEXT1, Article 8, (Mar. 2026). doi:10.1145/3786291.

[29] Jiayu Zhu and Yih-Chun Hu. 2026. vRecord dataset. Github. (2026). https://github.com/jimmyzhu12/vRecord.